



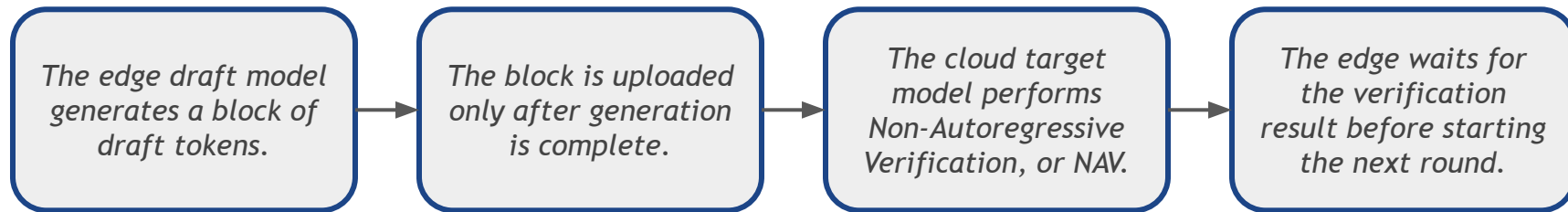
浙江大學
ZHEJIANG UNIVERSITY

PipeSD

An Efficient Cloud-Edge Collaborative Pipeline Inference
Framework with Speculative Decoding

Jiale Wang

Why Conventional Cloud-Edge Speculative Decoding Is Still Slow



Two Main Bottlenecks

- Serial generation and communication:** the edge processor is idle during upload, while the network is idle during draft generation.
- Inflexible fixed-length or single-threshold NAV triggering:** triggering too early repeatedly pays communication startup cost, while triggering too late accumulates low-quality tokens and increases rollback.

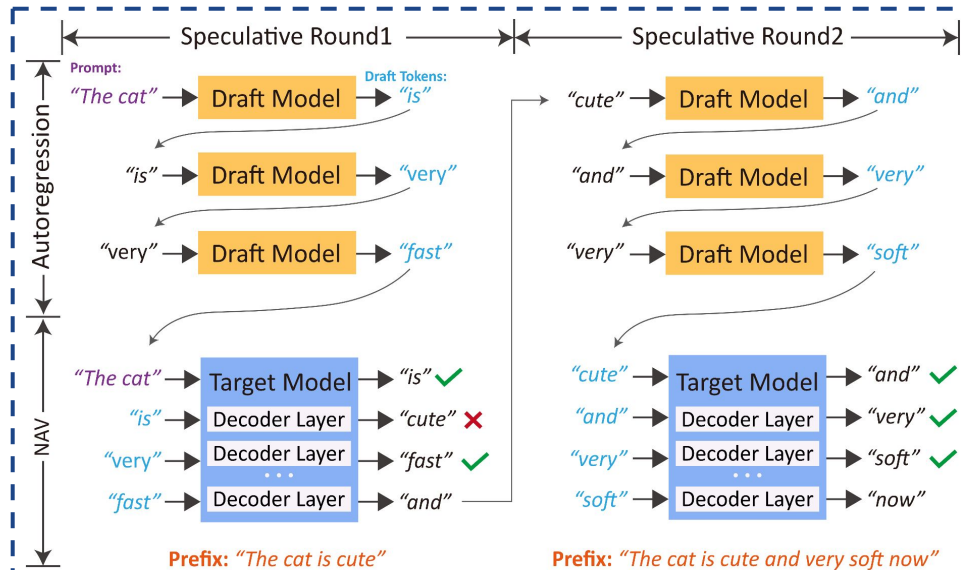
NAV (Non-Autoregressive Verification)



Let the edge draft model generate the candidate sequence $D=(d_1, d_2, \dots, d_{\square})$.

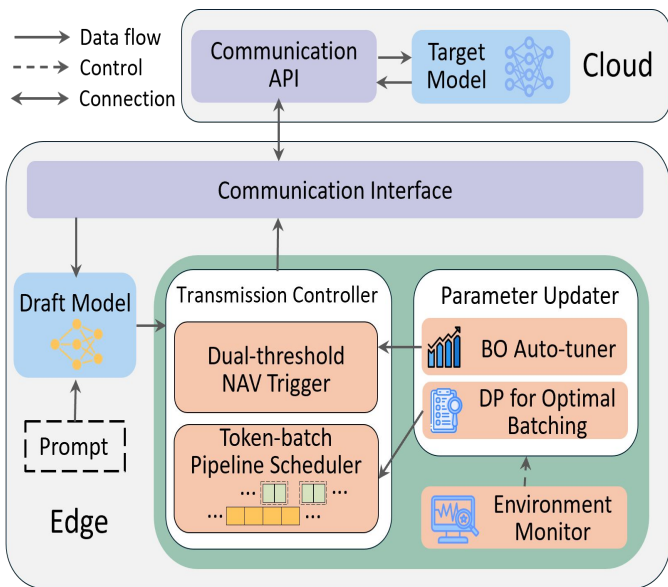
The cloud target model checks this sequence with one parallel forward pass:

- It accepts the longest prefix that satisfies the target-model verification rule.
- It produces one additional target token after the accepted prefix.
- If a rejection occurs, all draft tokens after the rejection point become invalid.
- The context of the next round must be based on the cloud-confirmed sequence.



Critical Reuse Condition: Edge-side work generated in advance can be reused only if the target model accepts the entire parent draft and its additional target token matches the token assumed by the proactive branch.

PipeSD Architecture



Communication API: receives candidate batches and their round metadata.
Target Model: performs NAV and returns the accepted prefix plus one additional target token.

Draft Model: autoregressively produces candidate tokens.

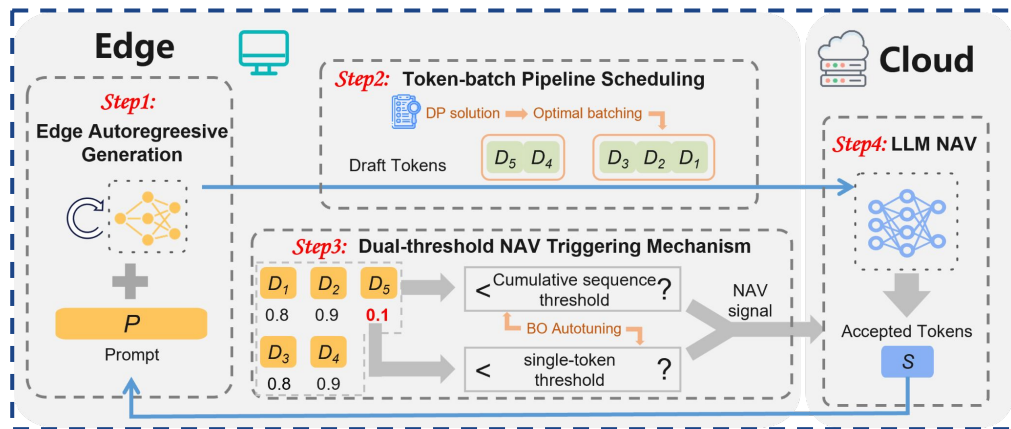
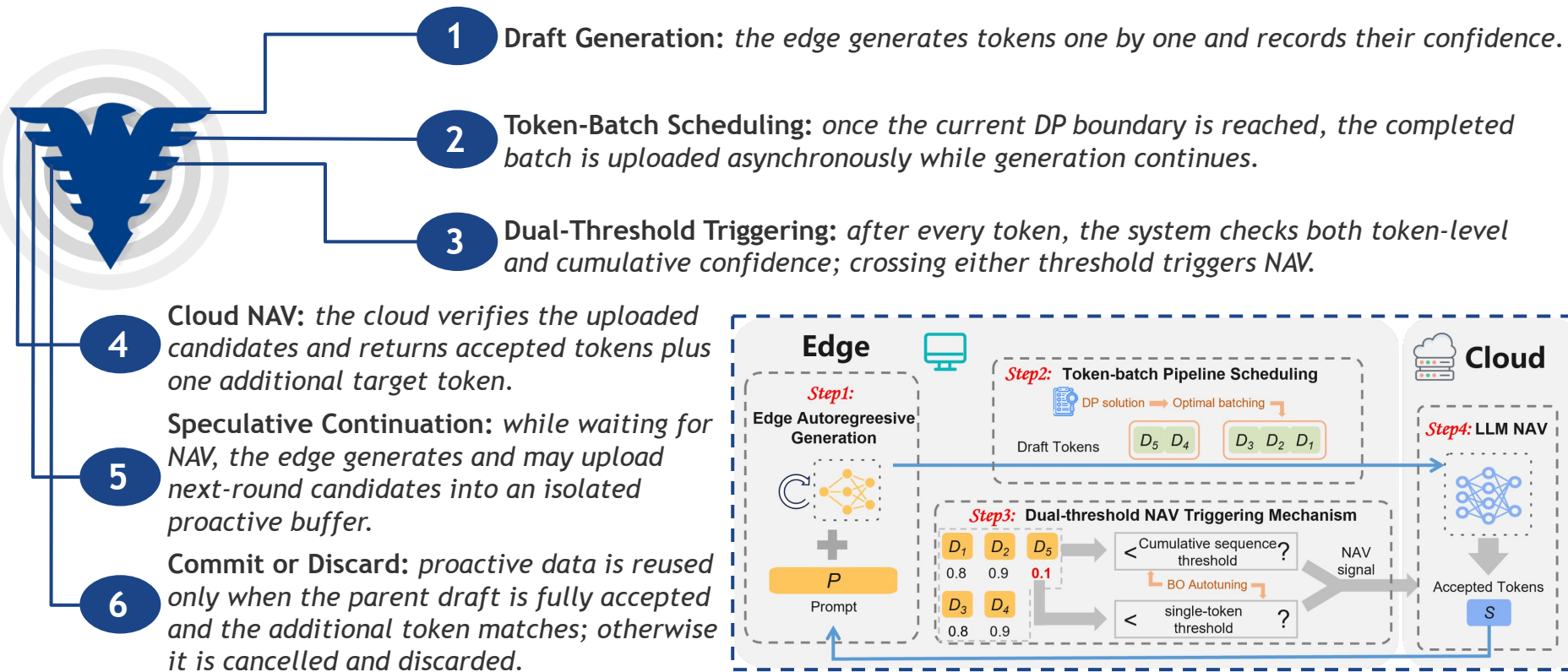
Transmission Controller: combines the **DP** batch scheduler and the dual-threshold NAV trigger.

Environment Monitor: estimates token-generation time (γ), communication startup cost (α), and per-token communication time (β).

Parameter Updater: uses **Bayesian optimization** to update the thresholds and recomputes DP batch boundaries when the environment changes.

Communication Interface: uploads batches and receives NAV results.

Pipeline Workflow



DP Scheduling



The paper models the transmission time and the generation time of a batch of size (**b**) as:

$$T_{comm}(b) = \alpha + \beta b, \quad T_{gen}(b) = \gamma b.$$

For a predicted window of **N** tokens, select batch boundaries that minimize the completion time of the final transmission.

$$dp[j] = \min_{0 \leq i < j} [\max(dp[i], \gamma j) + \alpha + \beta(j - i)]$$

- A batch can be sent only after all tokens in that batch have been generated and the previous transmission has **completed**.
- Dynamic programming enumerates the previous partition point with complexity ($O(N^2)$).
- (**N**) starts at 20 and is later updated using the moving average of draft lengths from the most recent 100 rounds.
- If NAV returns early, the active schedule is interrupted and unsent tokens are **cleared**.

BO Threshold



For the (n)-th draft token:

- Token-level confidence: $P(d_n)$
- Cumulative sequence confidence: $C_n = \prod_{i=1}^n P(d_i)$

NAV is triggered when: $C_n \leq R_1$ or $P(d_n) \leq R_2$.

Bayesian optimization searches for thresholds that minimize average time per token (TPT).

$$(R_1^*, R_2^*) = \underset{(R_1, R_2) \in (0,1)^2}{\operatorname{argmin}} \operatorname{TPT}(R_1, R_2)$$

- The **token threshold** detects a sudden local confidence drop.
- The **cumulative threshold** detects the growing risk created by several individually plausible tokens.
- The current implementation follows 16 budget and uses Expected Improvement with $\xi = 0.1$.
- PipeSD runs BO once at startup to initialize (R_1, R_2) , then **asynchronously** re-optimizes the thresholds when performance changes significantly, enabling online adaptation.

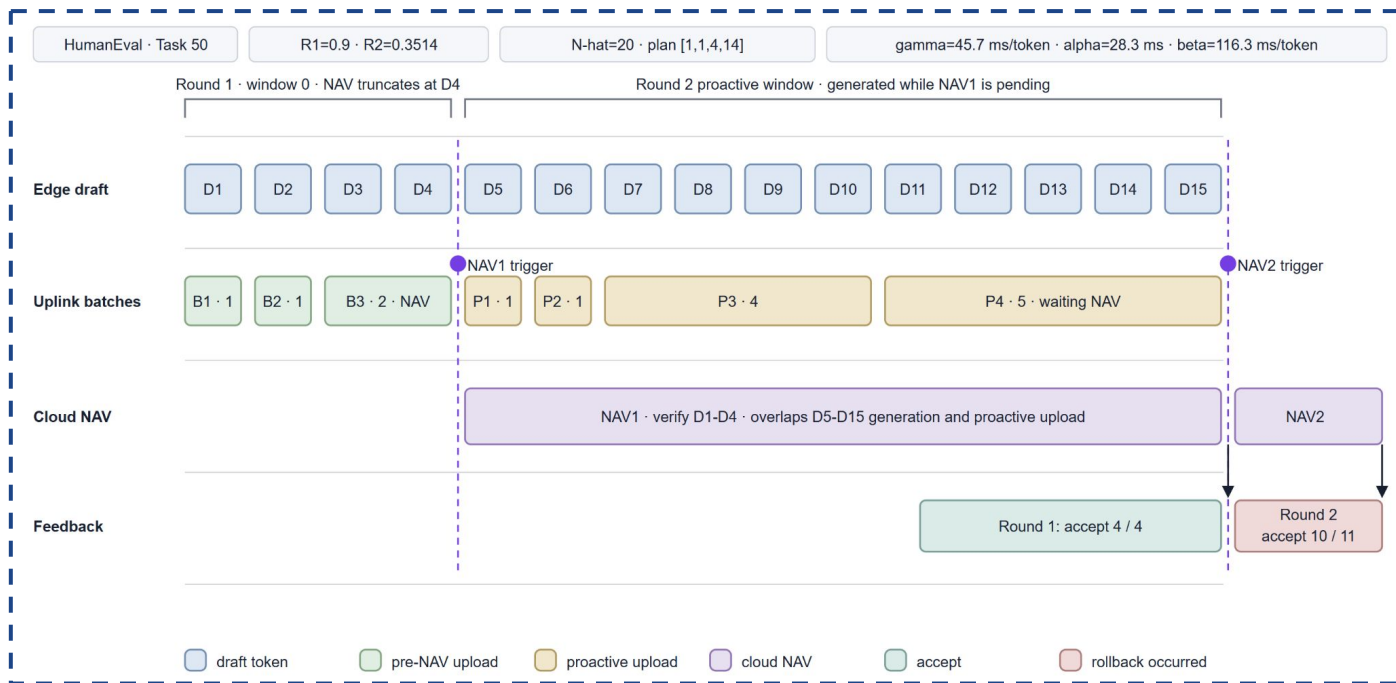
Data Lifecycle



Cloud Verification Result	Required Action
Only <i>part</i> of the parent draft is accepted	Roll back tokens after the rejection point and <i>cancel or discard</i> next-round proactive data
The <i>full</i> draft is accepted, but the additional target token does <i>not match</i>	The proactive context is incorrect; discard it and <i>restart</i> from the target token
The <i>full</i> draft is accepted and the additional token <i>matches</i>	Promote the isolated <i>proactive buffer</i> to the next formal round

Once proactive data is deemed invalid, its cleanup depends on its transmission stage. Tokens still stored locally are *discarded* immediately, queued batches are *cancelled* on a best-effort basis, while in-flight or cloud-buffered batches can no longer be retracted and must be discarded through *cloud-side* parent-round *validation*.

An Example



Reproduction Work



Role	Reproduction Setup	Model
Cloud	NVIDIA RTX PRO 6000	HumanEval: DeepSeek-Coder-6.7B GSM8K: Llama-2-7B-Chat
Edge	CPU (n_gpu_layers=0)	HumanEval: DeepSeek-Coder-1.3B GSM8K: TinyLlama-1.1B-Chat-v1.0

Table 1 Scenario 1

- **Network:** *SoftwareLink* emulates the paper's 20/200 Mbps link as **2.5 MB/s** uplink and **25 MB/s** downlink, while retaining real loopback HTTP and target-compute latency.
- **Link Contention:** Uplink and downlink use **independent FIFOs**; primary and proactive uploads share one uplink queue and therefore compete for the same 20 Mbps resource.
- **DP Scheduling:** Initial values are $a=25\text{ ms}$, $B=0.29/2.5=0.116\text{ s/token}$, $\gamma=0.036\text{ s/token}$, and $N=20$. Runtime estimates trigger batch replanning when they **change** by more than **20%**.
- **BO:** $R1/R2$ are searched once before formal evaluation and **remain fixed** during inference; online BO threshold updates are not yet implemented.
- **Evaluation:** Seed 3407, at most 128 generated tokens per sample, and exactly **1,000** cloud-accepted tokens per method. TPT is total end-to-end time divided by accepted tokens; NVML samples GPU power every 5 ms for prompt prefill and target NAV only.

Baselines

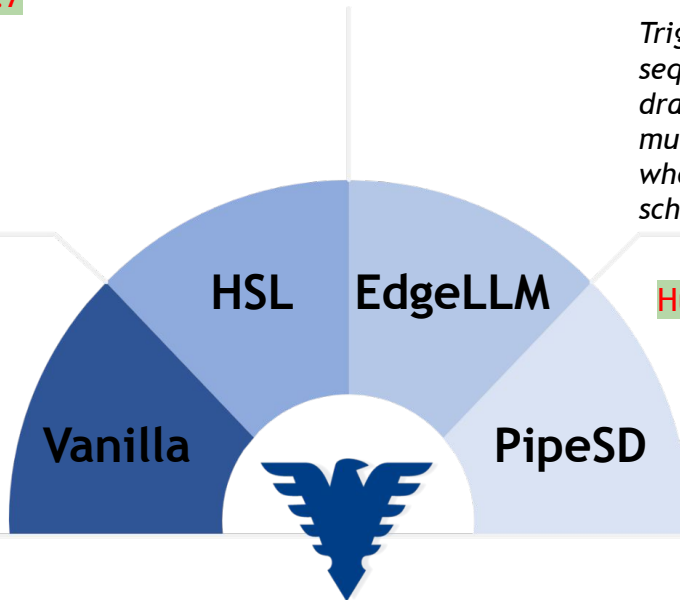


HumanEval:0.95 GSM8K:0.7

Triggers NAV when a token's confidence falls below a threshold, uploads the full block and pauses generation while waiting.

HumanEval:6 GSM8K:4

Generates a fixed number of draft tokens, uploads the whole block, and waits for NAV without overlap.



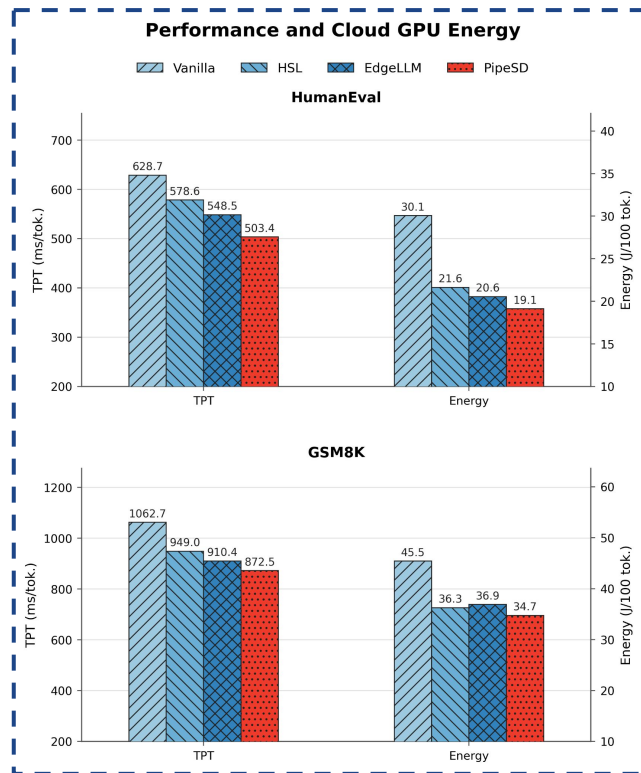
HumanEval:0.92 GSM8K:0.5

Triggers NAV using cumulative sequence confidence, continues drafting during verification, and multiplies the threshold by 0.8 when accepted tokens reach the scheduling window.

HumanEval:(0.9,0.35) GSM8K:(0.64,0.4)

Combines token and sequence-level thresholds with DP-based batch pipelining, continuing generation during NAV with safe reuse or rollback.

Evaluation Results



Task / Method	TPT (ms/ token)	Energy (J/100 tokens)
HumanEval Vanilla	628.705	30.060
HumanEval HSL	578.558	21.643
HumanEval EdgeLLM	548.539	20.552
<u>HumanEval PipeSD</u>	<u>503.444</u>	<u>19.121</u>
GSM8K Vanilla	1062.706	45.453
GSM8K HSL	949.016	36.254
GSM8K EdgeLLM	910.411	36.900
<u>GSM8K PipeSD</u>	<u>872.472</u>	<u>34.743</u>

Evaluation Results



Task / Method	Draft Length	Draft Tokens	Accept Rate	Total NAVs	Rollback Rate	Discard Rate
HumanEval Vanilla	5.881	1388	72.0%	236	42.8%	—
HumanEval HSL	4.408	1058	94.5%	240	20.8%	—
HumanEval EdgeLLM	5.841	1250	80.0%	214	31.3%	46.9%
<u>HumanEval PipeSD</u>	<u>5.279</u>	<u>1077</u>	<u>92.9%</u>	<u>204</u>	<u>24.0%</u>	<u>25.9%</u>
GSM8K Vanilla	3.797	1815	55.1%	478	60.9%	—
GSM8K HSL	2.998	1361	73.5%	454	46.5%	—
GSM8K EdgeLLM	2.708	1263	79.2%	466	36.1%	50.0%
<u>GSM8K PipeSD</u>	<u>2.995</u>	<u>1261</u>	<u>79.3%</u>	<u>421</u>	<u>46.3%</u>	<u>53.1%</u>

Rollback Rate is the number of NAV calls that reject at least one draft token divided by the total NAV count. It measures the fraction of NAV rounds with a rollback, not the fraction of rejected tokens.

Discard Rate = $\text{Discard} / (\text{Promoted} + \text{Discard})$, where Promoted denotes proactive tokens **promoted** to the next-round candidate, while Discard denotes proactive tokens **invalidated** by parent verification failure or an extra-token mismatch.

Four Modes

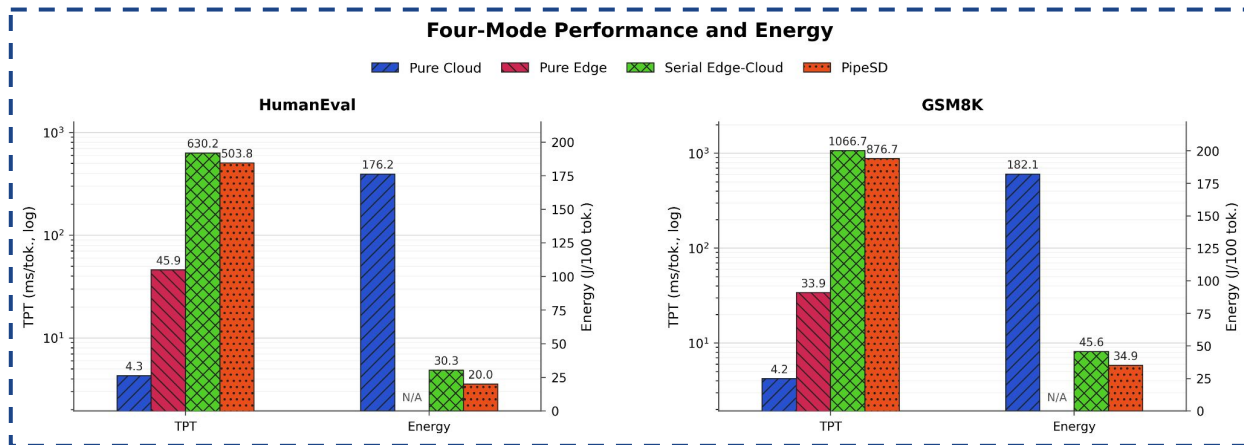


Pure Cloud: The target model performs prompt prefill and full autoregressive decoding on one GPU without network transfer or NAV, and energy covers only this GPU computation.

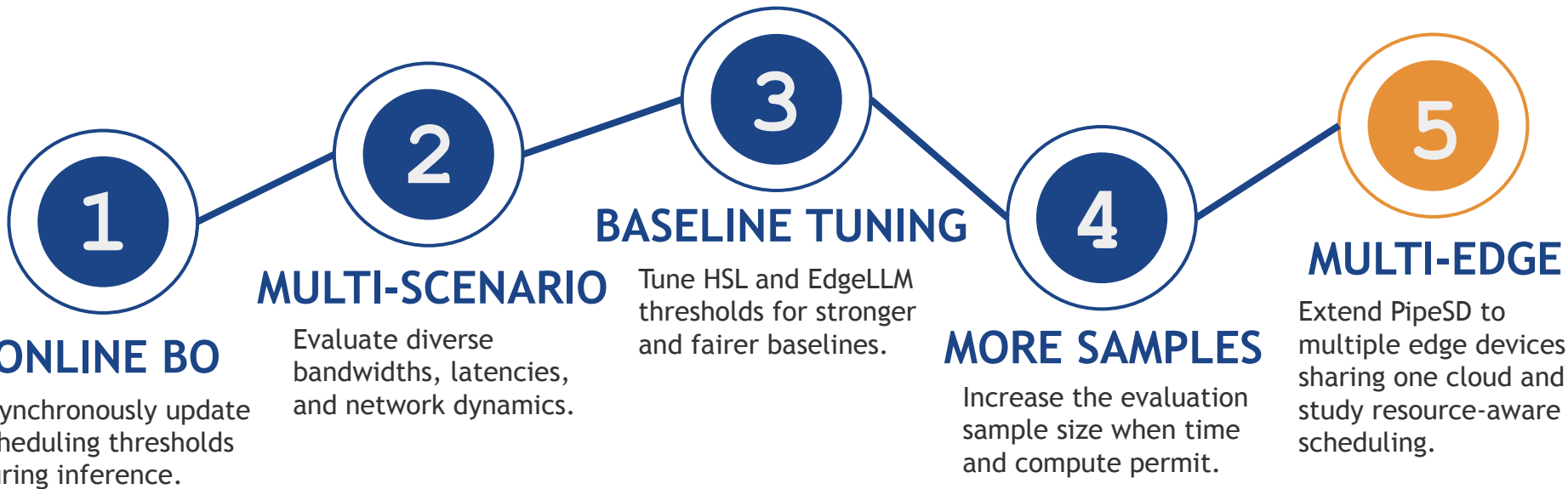
Pure Edge: The draft model performs standalone autoregressive decoding on the CPU without network, cloud, or NAV, while energy is unavailable due to missing RAPL access.

Serial Edge-Cloud SD: The CPU draft generates and uploads each fixed window before the GPU target performs NAV sequentially, with energy covering cloud prefill and active NAV computation.

PipeSD: PipeSD uses the same draft/target models and link, overlaps generation with DP-batched transfer, and continues drafting during NAV waits, with energy likewise covering cloud prefill and active NAV computation.



Plan





浙江大學
ZHEJIANG UNIVERSITY

Thank you for listening

